Using FSMs to Find Patterns for Off-Chain Computing

Finding Patterns for Off-Chain Computing with FSMs

Peter Bodorik
Faculty of Computer Science,
Dalhousie University
Peter.Bodorik@dal.ca

Christian, G., Liu
Faculty of Computer Science,
Dalhousie University
Chris.Liu@dal.ca

Dawn Jutla
Sobey School of Business, Saint
Mary's University
Dawn.Jutla@gmail.com

ABSTRACT

One of the problems arising in using blockchains is their size-constraints regarding performance. This paper proposes a new algorithm for blockchain software developers and architects to use for determining what computations of a smart contract can be effectively done off-chain without loss of trust. Our algorithm uses FSMs or HSMs in order to create smart contract patterns using graphs and then uses pattern recognition to identify which parts of the smart contracts should be considered for moving off-chain. The pattern recognition property used is that once software execution transits into the pattern's entry state, off-chain execution continues until the execution transits into the pattern's exit state, when execution continues on-chain. The software developer reviews each pattern together with information, such as anticipated overhead cost due to off-chain execution, and she is provided with guidance on decision making whether to execute the smart contract pattern under consideration off-chain. Expert software developer inspection, in the context of a Trade Finance use case, validates that our algorithm finds optimal patterns for moving computations off-chain and improve blockchain software performance.

CCS CONCEPTS

• **Computing methodologies** → Distributed computing methodologies; • **Networks** → Network services; • **Computer systems organization** → Architectures; • **Software and its engineering** → Software organization and properties; • **Information systems** → Information systems applications; Data management systems.

KEYWORDS

Blockchain, Smart contract, FSM and HSM modeling, Off-chain computation, Blockchain developer productivity

ACM Reference Format:
Peter Bodorik, Christian, G., Liu, and Dawn Jutla. 2021. Using FSMs to Find Patterns for Off-Chain Computing: Finding Patterns for Off-Chain Computing with FSMs. In *2021 The 3rd International Conference on Blockchain Technology (ICBCT '21), March 26–28, 2021, Shanghai, China.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3460537.3460565

ICBCT '21, March 26–28, 2021, Shanghai, China

ACM ISBN 978-1-4503-8962-4/21/03…$15.00
https://doi.org/10.1145/3460537.3460565

1 INTRODUCTION

One of the problems in use of blockchains is their ever-growing size and research and software development communities have strived to create models and techniques for moving computation and storage off the blockchain without losing the desirable blockchain properties. In this paper, we use the term off-chaining to refer to moving both computation and storage off the chain where it is performed using either side-chains or some other mechanisms. Clearly, the size of blockchains is important not only due to the fact that a blockchain has a replicated ledger, but also due to the associated increase in execution delays/costs of smart contracts methods and consensus mechanism used to check correctness of transactions in a block to be appended to the blockchain. For applications that use public blockchains, such as Ethereum or Bitcoin, the size is also very much a part of the actual dollar cost of executing smart contracts on a blockchain.

Eberhardt and Tai [2, 3] categorized methods, for reducing the blockchain size, based on what is moved to off-chain into: (i) off-chain storage, (ii) off-chain computation, and (iii) a hybrid approach, in which both storage and computation are off-chained. Off-chain storage is a well-established method in which data is stored off-chain in streams to which nodes subscribe and has been adopted, for instance, in [5]. All off-chain storage methods store an object off-chain but also store the object's hash code on the blockchain. Any time an object is retrieved from its off-chain storage, its hash-code is calculated and compared with the object's hash-code stored on the blockchain to ensure that the immutability property has not been violated. We also assume that all storage of objects and documents are stored off-chain using this mechanism and, thus we concentrate on the hybrid approach in which computation is also off-chain.

In [12], computation may be performed off-chain while proofing techniques are used to ensure that the blockchain trust properties are not undermined. Poon and Buterin [10] describe the Plasma project in which computation is performed off-chain while the blockchain is used for proofing techniques used for certification and storage of results of computation that are performed off-chain – the main chain is used for recording on the main blockchain only information, such as signatures, for results of proofs that the off-chain storage/computation was trustful. How to achieve attestation of results produced by off-chain processing are not described.

1.1 Our Approach

A smart contract may be considered as a collection of methods, which are supported by various data structures, to automate or transform use scenarios in a blockchain application. To determine which part(s) of a smart contract should be processed off-chain, it

must be determined which collection of methods and supporting data are to be processed off-chain. It is clear that selecting a random collection of methods to be processed off-chain is likely to be less suitable for processing than a collection of methods that collaborate to achieve a common goal and we shall refer to such a collection of methods as a pattern. Hence, when we refer to a *pattern* to be processed off-chain, we are referring to a collection of methods and supporting data to be processed off-chain.

We break down the problem of processing off-chains into three relatively independent subproblems:

- The first issue deals with determining which pattern should be processed off-chain – which is the main topic of this paper. Is one pattern more suitable to be processed off-chain than another, and if so, how do we find a pattern to be processed off-chain?
- For a pattern to be processed off-chain, it needs have two properties:
  a. One property deals with the issue of overhead costs. When a pattern is processed off-chain, there is also overhead cost introduced, on the blockchain, due to communication/interaction between on and off-chain processing – a pattern should be processed off-chain only if benefits outweigh overhead costs.
  b. Processing a pattern off-chain should not diminish the beneficial blockchain properties, such as resilience to failures or security attacks and immutability of blockchain data.
- The final issue is how to facilitate interfaces automatically for interaction between on and off-chain processing.

Broadly speaking, the three issues may be referred to briefly as "What", "When", and "How", respectively: (a) What is to be processed off-chain, i.e., which pattern should be considered for processing off-chain; (b) When processing off-chain should be facilitated, i.e., which conditions need to be satisfied by a pattern to be moved off-chain; and the third one is (c) How to facilitate automatically the interaction between on and off-chain processing.

In addition to addressing the above problems, our objective is to use separation of concerns in the design and development process by first allowing the smart contract developer to design the smart contract to satisfy the user/application requirements. Only then does the developer, or architect, use our approach to determine which smart-contract patterns should be considered for off-chain processing use, under which conditions, and how the on and off-chain processing should interact. If the developer needs to be concerned at the same time about application requirements and patterns to process off-chain, the problem complexity increases greatly.

1.2 Objectives

Our main objective for this paper ais to provide a new algorithm method and its validation for solving the first issue listed above dealing with determining which smart contract patterns should be processed off-chain. A second objective is to describe our approach to solving the other two associated issues listed above.

1.3 Outline

The section 2 describes background research results that we use. The section 3 describes our approach to finding patterns to be processed off-chain. The section 4 outlines our approach of: (i) How to create interface between on and off-chain processing automatically and (ii) how to evaluate a pattern in deciding whether it should be processed off-chain in terms of its overhead vs. benefits and in terms of mitigating risks in lowering trust when processing off-chain is used. The final section provides a summary and concluding remarks.

2 BACKGROUND

We briefly review Finite State Machines (FSMs) and also Hierarchical State Machines (HSMs) that are used to model blockchains. As we also use both concepts here, we briefly review their notation. Following this we describe modeling and use of FSMs for ensuring security of smart contracts [8, 9].

FSMs have been used for a long time for modeling of transactions and applications in many domains, including business and commerce. As smart contracts execution on blockchains includes state data/variables that are stored on the blockchain, they are suitable for modeling of smart contracts using FSM – in fact blockchains have been referred to as state machines [11]. As we also use FSMs to model blockchain smart contracts, we briefly review the FSM notation adopted from [8] with the exception of guards and labels that we omit for simplicity in presentation: An FSM F can be described as F = (S, s_0, T, I, O), where:

- S: Set of n states { s_0, s_1, . . ., s_{n-1} }, where s0 is the initial state of the FSM
- T: Set of transitions { t_i, i=1, 2, . . ., m }, such that each t_i has two components, t_i.from and t_i.to, that identify the transition to be from the state t_i.from to the state t_i.to, respectively
- I: Set of inputs { x_i }. Each transition t_i has input x_i that causes the transition. (x_i may be a large object or a set of objects)
- O: Set of outputs { y_j }. Each transition t_j has a set of outputs where each y_j may be a single value or it may be an object or a set of objects, whose value is produced by the state activities and is output by the state transition.

We note that S is a connected graph and, furthermore, with the exception of the start and final nodes, we assume that each node has at least one incoming and at least one outgoing edge (otherwise the FSM is not considered to be well-formed).

In late 80's, FSMs have been extended with the concept of hierarchy, leading to HSMs that can contain states that are themselves other FSMs. As we also utilize the concept of HSM, we review it briefly. An HSM can be defined using induction as follows [13]. In the base case, an FSM is a hierarchical machine. Suppose that M is a set of HSMs. If F is an FSM with a set of states, S, and there is a mapping function f: S $\mapsto$ M, then the triple (F, M, f) is an HSM. Any HSM has a corresponding equivalent FSM that can be achieved by "flattening" the hierarchy in HSM: Each state, s Ω S, that represents an HSM is replaced by its mapping (f(s)). HSMs recognize the same language as its corresponding flattened FSM. HSMs do not increase expressiveness of FSMs, only succinctness in representing them.

Our work draws upon research results described in [8, 9], which uses separation of concerns between designing a smart contract to

satisfy the user requirements and the design for security purposes. They model the smart contract using FSM modeling and then translate the model into a set of methods with a help of using a tool that ensures that security concerns are also satisfied.

We note that patterns in software and smart contracts have played a major role in the design and implementation of blockchain smart contracts. Examples of such patterns were identified in [4], which include Fail early and Fail loud, State machine, Upgradable registry, Transition counter and other patterns. Additional examples of patterns include the Challenge-response and Chess-end-game patterns in [3], letter of credit [7], and a Blind-bidding pattern in [8]. They also include patterns used for mitigating various security issues, such as a locking pattern to prevent re-entrance and access-control pattern in programming of smart contracts [8, 9].

3 FINDING PATTERNS TO PROCESS OFF-CHAIN

To find which smart contract patterns should be processed off-chain we examined many patterns and investigated various methods to recognize/find them. After trying a number of methods, we zeroed in on using FSM modeling of a smart contract and then analyzing the directed FSM graph of the pattern, where the FSM graph consists of nodes that represent the states of the FSM and edges are transitions between the states and their direction is defined by the transitions between the states. We developed algorithms to find/identify many patterns. However, we found that our algorithms to automatically identify a pattern failed if a pattern had minor variations – a problem arising due to the State Explosion problem with FSMs. For instance, variations in approval processes lead to different FSMs with different number of states and connections. To solve for this, instead of using a shape of a pattern (represented by a subgraph of an FSM graph) for recognition purposes, we should define and enforce certain properties. We propose a pattern should have a property that represents cooperation/coordination of activities of actors that lead to one common goal, activities that are independent of activities for other goals. This leads to a complementary property in that once the pattern's execution starts, then all subsequent invocations of the method should be for the methods of the same pattern until the execution of the pattern completes. If such a pattern is processed off-chain, then once its execution starts by invocation of a method that is a part of the pattern, all subsequent method invocations would also be only for methods processed off-chain until the pattern completes.

As transitions amongst the states are indicated by edges, we noted that state transitions, represented in the graphs by edges, had mostly internal edges amongst nodes representing the pattern. It led us to consider that a pattern should have one entry edge and one exit edge - representing one objective in pattern computation. We considered this property, for pattern recognition, but it too failed in the situation where the pattern may be invoked from different states. Also, the pattern may be repeated in an FSM graph as agreements may be required for different purposes, such as a 2-party agreement approval of a contract or an agreement on a price between seller and a buyer. We thus modified the property as described below and use it in a proposed algorithm to find patterns to be considered for processing off-chain. We present the properties below and, in the section 4, we provide an algorithm to find them.

3.1 Subgraph Properties for Processing Off-chain

Consider an FSM F, its set of states, S, and a subgraph S' $\subset$ S. Any edge (x, y), x, y $\in$ S', is called an *internal* edge of S', while it is called an *external* edge if x $\notin$ S' or y $\notin$ S'. Informally, an internal edge must connect two internal nodes of S. Relative to the subgraph S', an edge is external if either of its nodes is an external node, i.e., it is not in S'. We postulate that a connected subgraph S', which is a subset of S (i.e., S' $\subset$ S), that has the following properties should be considered for processing off-chain:

- All subgraph nodes, with the exception of entry and exit nodes, have only internal edges (x, y), where x, y $\in$ S'.
- There exists exactly one node/vertex, s_{en}, in the subgraph that, in addition to its internal edges, has any incoming edges from external nodes – such a node is referred to as an *entry* node. (An entry node does not have any outgoing edges to external nodes.)
- There exists exactly one node/vertex, s_{ex}, in the subgraph that, in addition to its internal edges, has any outgoing edges to any external nodes – such a node is referred to as an *exit* node. (An exit node does not have any incoming edges from external nodes.)

As a consequence of the above, with the exception of the entry and exit nodes, all internal nodes of the subgraph have edges connected only to other internal nodes. That a subgraph has only one entry and one exit node means that once the initial state of the subgraph is reached, it has only one specific purpose that is reached upon transition from the exit node of the subgraph. As a consequence of this property, if the simple subgraph with an entry node s_{en} and an exit node s_{ex}, transits into the s_{en} state/node, any subsequent invocations of the smart contract methods will be executed off-chain until the state of execution transits into the s_{ex} state, at which point any subsequent executions of the smart contract methods will be processed back on-chain. We refer to subgraphs, of an FSM graph, that satisfy the above properties as simple subgraphs.

We note that a simple subgraph is such that it may contain other simple subgraphs within it. A simple subgraph must be connected as otherwise, the state graph of the FSM would have unreachable states, as the simple subgraph, with the exception of the entry and exit nodes, has internal edges only. Consequently, if the subgraph has any disconnected node, that means that the state graph of the FSM would also have unreachable states.

In the following subsection we describe an algorithm that uses the above properties to find all simple subgraphs for a given FSM graph and another one that shows how it is used.

3.2 Finding and Using Simple Subgraph Patterns

We describe an algorithm that uses the simple subgraph properties to discover all simple subgraphs for a given FSM. Once simple

subgraphs are found, they are ordered and presented to the developer one after another for determination whether they should be executed off-chain: If a simple subgraph pattern is to be processed off-chain, then the FSM model is modified to represent the off-chain processing.

It should be noted that, for simplicity in presentation and ease of understanding, we describe a straight-forward brute-force algorithm in a semi-formal manner rather than concentrating on efficiency and formalism. Furthermore, the algorithm is used in the implementation design process only, which is a one-time task, rather than in the execution of a smart contract once it is designed and developed. For similar reasons, we do not provide details for the following methods that are used by the algorithm and in which FSM F description is assumed to be global:

- boolean isSimpleSubgraph (parameter: set S') . . . the method determines whether S' is a proper subset of S (S' ⊂ S).
- boolean isProperSubgraph (parameters: set S_1, set S_2) . . . the method returns true if $S_1 \subset S_2$, i.e., if S_1 is a proper subset of S_2, where both S_1 and S_2 are proper subgraphs of S.
- set generateAllSubgraps (parameter FSM F) . . . the method has as input FSM F = (S, s_0, T, I, O) and returns a set of elements where each element is a simple subgraph of S.

Algorithm 1, to find simple subgraphs, is shown below. Note that after all subgraphs are generated (line 2 in Algorithm 1), it is determined for each one if it is a simple subgraph (line 5). If it is, it is stored in a set of simple subgraphs (line 7). Lines 9-12 find out, for each simple subgraph Sx, how many other simple subgraphs (Sx.count) it contains - next we describe how Sx.count is determined and used when we discuss Algorithm 2.

Algorithm 1 – find all simple subgraphs for an FSM F

Input: *FSM F* = (S, s0, T, I, O)
Output: *Set Y* = { S1, S2, . . ., Sn } . . . set of all simple subgraphs S1, S2, . . ., Sn of FSM F. Each Si ∈ Y has an attribute Si.count, which is the number of other simple subgraphs Sj, i ≠ j, in Y that are subsets of Si, i.e., where Sj ∈ Si

```
1.   //  Each graph S has an attribute S.count that denotes the number simple subgraphs it has
2.   Set X = generateAllSubgraps (FSM F);
3.   Set Y = NULL;
4.   for each subgraph S' in X
5.   if ( isSimpleSubgraph (S') ) then {6.   S'.count=0;
7.    Y.add(S')
8.   };
9.   for each Sx in L {10.      for each Sy in L
11.          if ( isProperSubset (Sy, Sx) ) then += Sx.count;
12.  }
```

Algorithm 2, shown below, has a list of simple subgraphs as its input, where each subgraph identifies a pattern. They are presented to the developer, together with additional information, for the final decision making on which of the patterns should be processed off-chain. We shall discuss this decision-making problem further in the following section. Here we highlight that the subgraphs are presented in the descending order as defined by S.count that denotes the number of other simple subgraphs that a simple subgraph has. The reason for that is that, if a pattern, represented by a subgraph S_x, is processed off-chain, then, any pattern represented by simple-subgraph S_y, which is a subset of S_x, is also processed/executed off-chain automatically as a part of execution of S_x.

Algorithm 2 – Iteratively present to the developer simple subgraphs with information for off-chaining consideration

Input: *FSM F* = (S, s0, T, I, O) and
Set Y = {S1, S2, . . ., Sn } . . . set of all simple subgraphs S1, S2, . . ., Sn of FSM F found for F using Algorithm 1
Output: *HSM H* that is derived from FSM F with off-chained computations (simple subgraphs) represented with nodes that are HSMs.
List of simple subgraphs L = {S1, S2, . . . } of which computation is to be performed off-chain. L is in descending order by Si.count.

```
1.  ordered list L = orderSetOfSimpleSubgraphs (set Y)
2.  list of simple subgraphs L = ( null )
3.  initialize HSM H to be initially FSM F;
4.  for each Sx in L {5.  present to the developer subgraph Sx and related information;
6.  if ( developer decides to off-chain Sx ) {7.  replace the subgraph Sx in FSM F with an HSM representing Sx;
8.      L.add (Sx);
9.          }
10. }
```

We make a note about the statement on line 10 of Algorithm 1, line that is used to count the number of simple subgraphs that a simple subgraph has. The statement exploits the following the theorem 1.

Theorem 1: Consider an FSM F, with a set of states, S, that has two simple subgraphs, S1 ⊂ S and S2 ⊂ S, where |S1| > 1 and |S2| > 1. Then one of the following holds:

(a) S1 ⊂ S2 or S2 ⊂ S1
(b) S1 and S2 are such that they either
  (i) share nodes that form a simple subgraph or
  (ii) that they share at most one node and, furthermore, that the shared node is such that it is the entry node for one of S1 and S2 and an exit node for the other one.

We use the above property in counting the number of other simple subgraphs that a simple subgraph has. Due to space limitations, we do not provide the proof, but the proof can be found in [6].

Once the subgraphs are identified, Algorithm 2 is used to present the developer with one pattern at a time to be considered for processing off-chain. Also provided is information required for the decision making as described in the following section. Once a subgraph is slated to be processed off-chain, it is replaced in the FSM graph with its HSM representation. For instance, assume that the escrow deposit pattern, shown in Figure 1, is a part of an FSM graph. If it is decided to process the pattern off-chain, the pattern is replaced by its HSM node representation in the FSM graph, as is shown in Figure 2.

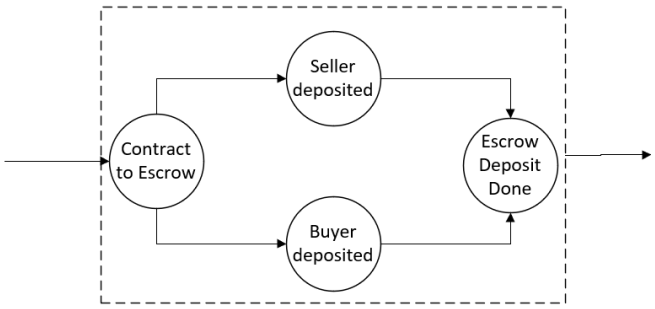

Figure 1: Escrow deposit pattern represented by a simple subgraph, of a larger smart contract graph, before decision to off-chain

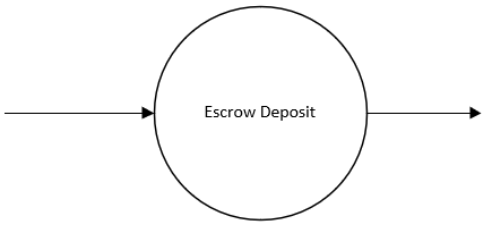

Figure 2: Simple subgraph represented by HSM node as a part of a smart contact graph

3.3 Finding Simple Subgraphs: Buyer-Seller Interaction with Escrow Deposit Use Case

We used use cases to validate that our algorithm would find the patterns of interest and what other kinds of patterns it would find. We show results of applying Algorithm 1 on use cases found in the blockchain literature. We created an FSM model of a smart contract to govern interaction between actors and then used Algorithm 1 to find simple subgraphs. We first overview the use case, apply Algorithm 1, and then discuss the discovered simple subgraphs. It should be noted that we manually examined many use cases to determine various patterns in them that have been discussed in blockchain literature, patterns, such as sequence of events, escrow deposit, approval by actors.

The use case [1] involves a seller and a buyer of some product, wherein the case includes an escrow deposit amongst other patterns. After the product is posted for sale, the buyer and seller negotiate price. Once an agreement is reached on the price, a contract is prepared and signed that stipulates escrow deposit and the matter of delivery. Once each of the buyer and seller makes a deposit to an escrow account, then shipment, which includes crossing borders and hence involving customs, occurs that involves delivery to a port, going through customs, storing on a ship, then processed at the destination customs, unloaded from ship to port, buyer pickup, and finally followed by a return of escrow deposit. We created the FSM for this case and then applied Algorithm 1 to find all simple subgraphs. Figure 3 shows the graph of the FSM model and also simple subgraphs that were identified by Algorithm 1

We now discuss the simple subgraphs of Figure 3 in relation to the application semantics. Further discussion on information presented to the developer (such as the overhead cost for on-chain-to-off-chain interaction) when considering off-chain processing is discussed in the next section.

- *Simple subgraph S_1 ...* contains states representing negotiation to reach a price between the buyer and a seller. If semantics indicate that details on negotiations are not important, then the computation can be performed off-chain as long as upon return from off-chain processing, the negotiated price is signed by each party, as confirmation of the agreement on price, so that it is recorded on the blockchain.
- *S_2 ...* represents signing of the contract that details the price and shipment delivery. Upon return from off-chain processing, the contract that is signed by both the seller and buyer needs to be recorded on the blockchain.
- *S_3 ...* represents escrow deposits made by each of the seller and the buyer. Upon return, confirmation of both deposits from the escrow account holder needs to be recorded on the blockchain.
- *S_4 ...* represents a sequence of events involving distinct actors in the shipment. If processed off-chain, digitally signed information about the outcome of each of the states should be recorded on the blockchain.
- *S_5 ...* represents return of escrow deposits. If processed off-chain, digitally signed receipt about each of the return of funds should be recorded on the blockchain.

It should be noted that two *simple* subgraphs, where one is not a subset of the other, may be such that they share at most one node and, furthermore, that the shared node is an *entry* node for one of the subgraphs and an *exit* node in the other. For instance, simple subgraphs S_3 and S_4 share the node "Escrow Deposits Done" or simple subgraphs S_4 and S_5 share the node "Buyer Pickup".

There are additional simple subgraphs that were found, simple subgraphs that contained other simple subgraphs, such as S_6 shown in Figure 3 that contains subgraphs S_2 and S_3. There were other subgraphs that were found but they are not shown in the figure as it would get too busy.

Concluding this section, we note that we applied Algorithm 1 to find patterns for all use cases listed at the end of the section 2 plus on some use cases that were reported in the business literature. We observed that the algorithm found all patterns that we flagged through manual inspection as suitable to be processed off-chain if benefits outweigh the overhead costs, which is a subject of the next section. Furthermore, the algorithm also found patterns that we missed. However, the limitation of our work is that until we do years of exhaustive use case testing and validation, we cannot and do not claim that our algorithm finds universal patterns that may be suitable to be processed off-chain as we do not yet have an authoritative definition that describes all properties of subgraphs suitable for off-chaining across all use cases.

4 DECIDING TO PROCESS OFF-CHAIN AND INTERFACE

Clearly, a pattern should be processed off-chain only if the cost benefits outweigh those of overhead. Of course, to evaluate the

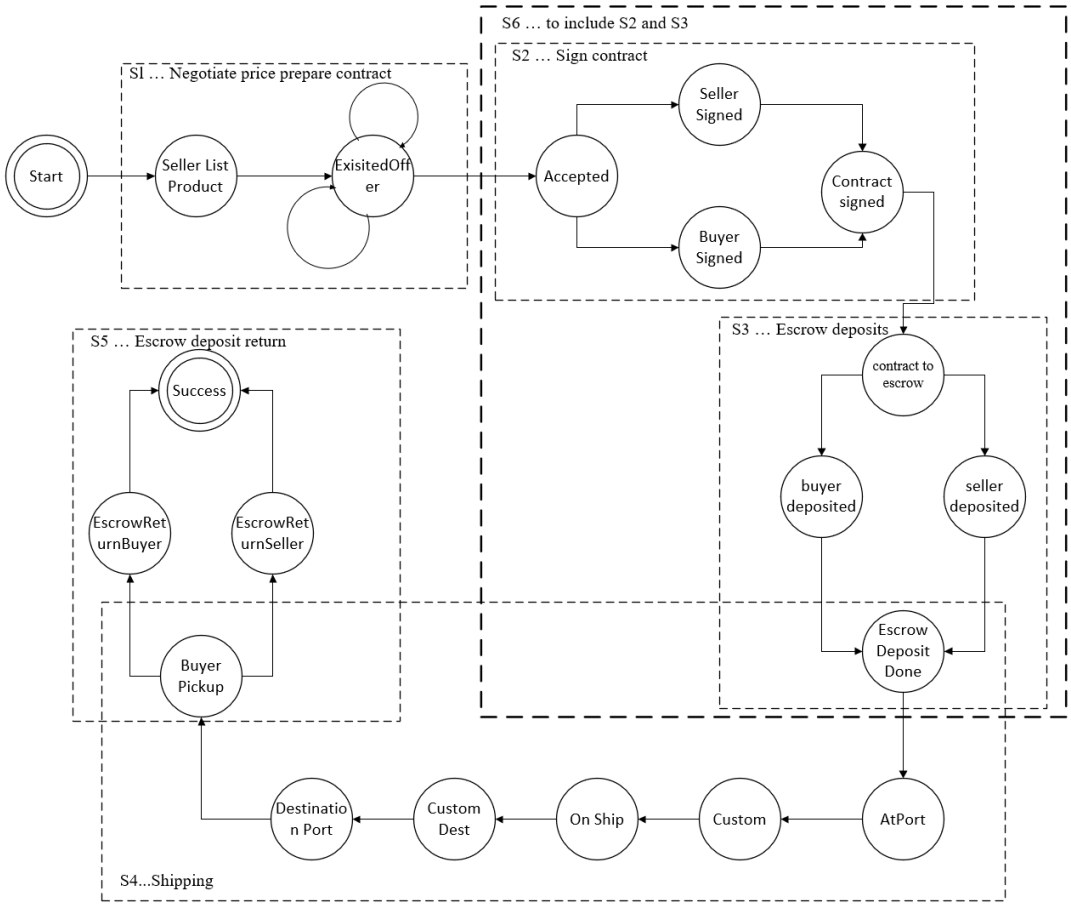

Figure 3: FSM graph for the buyer-seller interaction using escrow deposit and found simple subgraph patterns

costs, the interface between off and on-chain processing needs to be determined, which means that the question of how to create interface must be solved. Furthermore, a pattern should be processed off-chain if it does not negatively affect the trust in blockchain. We first discuss briefly our approach to mitigating lowering of trust due to off-chain processing and then we address the issue creating the interface automatically and how to evaluate costs.

4.1 Off-chain Computation and Blockchain Properties

Blockchains have a number of desirable properties and how blockchain properties are affected when deciding to process off-chain and, if risks arise, how they should be mitigated still needs to be investigated. Here, we describe briefly the properties of immutability, availability, but concentrate on the issues of trust.

Immutability: Immutability of data stored and used off-chain is safeguarded by using hashes. Data stored off-chain is signed with a hash-code that is stored on the blockchain and used to ascertain that data retrieved off-chain has not been modified. This is a standard technique for off-chain storage.

Availability: As a system availability is determined by the availability of its components, if computation is performed off-chain, its availability will affect the overall system availability. Hence, off-chain computation also needs to support availability. Fortunately, providing a desired availability via replication is a well-established and researched problem with good solutions.

Trust: This is the most difficult issue. Trust is gained by blockchain and its smart contracts because all parties can examine the smart contract code and changes to the state variables (in permissioned blockchains, as long access rights are available). As parts of the smart contract are moved off-chain, that may decrease trust on the part of the blockchain participants as questions may be raised regarding the off-chain processing and its implication on trust. To ensure trust in processing patterns off-chain, we provide the developer with supporting information garnered from the FSM definition as discussed below.

Our methods provide the software developer with information on all methods of the pattern considered for processing off-chain, together with information on writes to the blockchain. Upon the completion of the pattern processing off-chain, attestation methods executed off-chain are used to examine the results of off-chain

computation to ensure that they equivalent to results when the pattern is processed on chain. Of course, this increases overhead costs that may or may not exceed any benefits/savings for executing the method off-chain. Furthermore, we also provide the developer with a cost analysis of benefits vs overhead cost. However, as it involves evaluating the cost of the interface between on and off-chain processing, we postpone its discussion after the next subsection.

4.2 Interface between On and Off-chain Computation and Benefits vs Overhead Cost Analysis

For any pattern that is to be executed off-chain, we replace the code, of a method that is slated off-chain, with code that gathers appropriate information and provides it to the bridge process between on and off-chain communication. The bridge process also provides a cache for the blockchain reads and writes by off-chain processing. Any reads and writes by the off-chain method need to be performed on the cache provided by the bridge. The bridge process also stores all input and output parameters and state transitions. Any output parameters produced by the methods executed off-chain are received from the off-chain computation by the bridge process and communicated to the invoking application.

Special processing is performed when a method is invoked that corresponds to the entry state or exit state. When the pattern is entered, the bridge process seeds the off-chain cache with state variables (data stored on the blockchain) that the pattern needs, including the variable denoting the state of computation and any other blockchain data read by the method. Upon exit from computation, the bridge process harvests data from the cache information about all calls to methods, their inputs and outputs, transitions, and data written to the blockchain (actually written to the cache) by the off-chain methods. This data is provided to the attestation methods for the results of off-chain computation. If all is OK, the data used for attestation is recorded on the blockchain.

Details about the bridge process can be found in [6]. They include detailed descriptions of steps for invoking an off-chain method and also return from such a method, including steps taken when *entry* and *exit* methods are invoked. As we have the details of the bridge actions, we developed a cost model that evaluates the cost of overhead of the parts of the bridge that executes on the blockchain. This enables us to compare the cost of a method executed on the block with the overhead due to interface between on and off-chain execution. We used the cost for determining cost benefits vs overhead for smart-contract methods on the Ethereum blockchain [6].

5 SUMMARY AND CONCLUSIONS

Our new algorithm and its methods address the issue of a software developer finding parts of smart contracts to be moved for off-chain execution. We explored many patterns in search of those that are suitable for off-chaining and in search of an algorithm that would recognize them automatically. Our search led us to subgraph patterns' properties for off-chain computation and created an algorithm to find the patterns using those properties. We also reported testing the algorithm on smart contracts in a Trade Finance use case and a variety of smaller use cases. To the best of our knowledge, this is the first attempt at identifying "what" parts of the smart contracts should be selected for processing off-chain. Previous research concentrates on the question of "how" to accomplish off-chain processing, i.e., they address of how a given method, or a collection of methods, can be processed of chain, e.g., [14].

To ensure off-chain trustworthiness, our algorithm collects information on off-chain processing and providing it to methods that do attestation of results produced by off-chain computing as correct. Attestation methods, however, still comprise research problem that is likely to require the use of sematic information for its solution and to reduce attestation costs. Finally, we also report on automatically creating an interface between on and off-chain processing and the cost model to evaluate the trade-off between on and off-chain processing.

In conclusion, we validated our algorithm and methods using a complex Trade Finance use case, where expert blockchain software developers manually inspected and verified the identified patterns as optimal for off-chain computation. However, some of our further work will involve validation of our algorithm to universally assist the developer in deciding whether a pattern should be executed off-chain across a broader swathe of popular blockchain use cases.

REFERENCES

[1] Asgaonkar A. & Krishnamachar B. (2019). Solving the Buyer and Seller's Dilemma: A Dual-Deposit Escrow Smart Contract. IEEE Int. Conf. on Blockchain and Cryptocurrency (ICBC). 262-267.

[2] Eberhardt J. & Tai S. (2017). On or Off the Blockchain? In: De Paoli F., Schulte S., Broch Johnsen E. (eds) Service-Oriented and Cloud Computing. ESOCC 2017. Lecture Notes in CS, 3-15, vol 10465, Springer.

[3] Eberhardt J., Jonathan H. (2018) Off-chaining Models and Approaches to Off-chain Computations. In Proceedings of the 2nd Workshop on Scalable and Resilient Infrastructures for Distributed Ledgers (SERIAL'18). 7-12.

[4] HeartBank (2018). Smart Contract Design Patterns: A Case Study with Real Solidity Code. Retrieved from https://medium.com/heartbankstudio/smart-contract-design-patterns-8b7ca8b80dfb on Nov. 2, 2020.

[5] Lisk. (2020). What is Blockchain? Retrieved from https://lisk.io/what-is-blockchain on Nov 2, 2020.

[6] Liu, C. (2021). FSM for Modeling for Off-Blockchain Computation. Master of Computer Science Thesis. Dalhousie University, Halifax, Nova Scotia, Canada.

[7] Ledger Insights. (2018). Letter of Credit blockchain launches with eight banks. Retrieved from https://www.ledgerinsights.com/blockchain-letter-of-credit-trade-finance-voltron/ on Nov 8, 2020.

[8] Mavridou, A., & Laszka, A. (2018(a)). Designing Secure Ethereum Smart Contracts: A Finite State Machine Based Approach. Financial Cryptography. DOI: 10.1007/978-3-662-58387-6_28

[9] Mavridou A., Laszka A. (2018(b)) Tool Demonstration: FSolidM for Designing Secure Ethereum Smart Contracts. In: Bauer L., Kusters R. (eds) Principles of Security and Trust. POST 2018. Lecture Notes in Computer Science, vol. 10804. Springer. 270-277.

[10] Poon, J. & Buterin V. (2017). Plasma: Scalable Autonomous Smart Contracts. https://plasma.io/plasma.pdf on Nov. 12, 2020. 47 pages.

[11] Saito K. & Yamada H. (2016) What's So Different about Blockchain? Blockchain: Probabilistic State Machine. 2016 ICDCSW. 168-175.

[12] Teutsch, J. and Reitwieuner, C. (2019). A scalable verification solution for blockchains. Retrieved from https://people.cs.uchicago.edu/ ~ teutsch/papers/truebit.pdf on Nov. 8, 2020.

[13] Yannakakis M. (2000) Hierarchical State Machines. In: van Leeuwen J., Watanabe O., Hagiya M., Mosses P.D., Ito T. (eds) Theoretical Computer Science: Exploring New Frontiers of Theoretical Informatics. TCS 2000. Lecture Notes in CS, vol 1872. Springer. 315-330.

[14] Xdaichain.com. (n.d.) Cost Estimation on XDai. Retrieved from https://challenge.xdaichain.com/cost-estimates on Jan. 14, 2021.